

# LAPORAN PRAKTIKUM BIG DATA

## Praktikum 3



OLEH :

JARDEL POLIVIERA

2411533012

DOSEN PENGAMPU:

AFDHAL DINILHAK. M.Kom

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG 2026

Link repo :

[https://github.com/JardelPoliviera/BigData26\\_B\\_2411533012\\_JardelPoliviera](https://github.com/JardelPoliviera/BigData26_B_2411533012_JardelPoliviera)

Link gdrive :

<https://drive.google.com/drive/u/0/folders/1ocK0iCKvU6Nvh8WlRw7J3a-scAArmHYx>

## A. PENDAHULUAN

Analisis mobilitas perkotaan modern menuntut pemahaman yang mendalam mengenai pola pergerakan masyarakat setiap harinya. Salah satu indikator terbaik untuk mengukur aktivitas ini adalah data operasional taksi kuning yang merekam jutaan perjalanan penumpang secara rinci. Data rekaman perjalanan ini tidak sekadar berisi angka, melainkan menyimpan informasi berharga terkait jam sibuk, jarak tempuh, hingga fluktuasi permintaan layanan transportasi komersial di suatu kota.

Studi kasus ini berfokus pada perbandingan komprehensif antara dataset operasional taksi pada bulan Januari dan Februari pada tahun yang sama. Tujuan utama dari analisis komparatif ini adalah untuk mengidentifikasi apakah terdapat pergeseran tren perilaku komuter atau justru menunjukkan pola yang konsisten dari waktu ke waktu. Melalui serangkaian proses ekstraksi, pembersihan data, hingga pemetaan visual menggunakan bahasa pemrograman Python, tahapan analisis ini dirancang untuk menyingkap wawasan penting di balik tingginya volume transaksi harian..

## B. TUJUAN

1. Mahasiswa dapat menyiapkan notebook Colab yang siap pakai dan mendokumentasikan spesifikasi lingkungan kerjanya (Python version, library version, RAM, disk).
2. Mahasiswa dapat mengunduh dataset publik secara terprogram dan memeriksa integritas awalnya (ukuran file, jumlah baris, skema kolom).
3. Mahasiswa dapat mengukur pemakaian memori sebuah DataFrame dan menurunkannya melalui pemilihan kolom serta optimasi tipe data.
4. Mahasiswa dapat memproses file yang lebih besar dari RAM yang tersedia menggunakan teknik chunking
5. Mahasiswa dapat menghasilkan tabel agregasi (per jam, per metode pembayaran) dan menyimpannya dalam format CSV serta Parquet.
6. Mahasiswa dapat menjalankan kueri agregasi menggunakan PySpark local mode dan menjelaskan perbedaan konseptualnya dengan pandas.

## C. LANGKAH PRAKTIKUM

### 1. K-1: Inisialisasi dan Utilitas Pemantauan

```
import sys, time, os, json, sqlite3, gc
import pandas as pd, numpy as np, psutil

for nama in ["pandas", "numpy", "pyarrow", "requests", "sklearn", "sqlalchemy"]:
    try:
        mod = __import__(nama)
        print(f"{nama:11s}: {mod.__version__}")
    except ImportError:
        print(f"{nama:11s}: BELUM TERPASANG")

from google.colab import drive
drive.mount("/content/drive")

DIR_MENTAH = "/content/lapisan_mentah"
DIR_KURASI = "/content/lapisan_terkurasi"
DIR_SIMPAN = "/content/drive/MyDrive/Colab Notebook/Big Data/Praktikum3"

for d in (DIR_MENTAH, DIR_KURASI, DIR_SIMPAN):
    os.makedirs(d, exist_ok=True)

proses = psutil.Process(os.getpid())
catatan = []

def rss_mb():
    return proses.memory_info().rss / 1024**2

def ukur(label, fungsi):
    m0, t0 = rss_mb(), time.perf_counter()
    hasil = fungsi()
    detik = time.perf_counter() - t0
    catatan.append({
        "langkah": label,
        "detik": round(detik, 2),
        "delta_rss_mb": round(rss_mb() - m0, 1)
    })
    print(f"[{label}] {detik:.2f} s")
    return hasil

lineage = []

def catat_sumber(nama, asal, jumlah_baris, keterangan=""):
    lineage.append({
        "sumber": nama,
        "asal": asal,
        "diambil_pada": pd.Timestamp.now("UTC").isoformat(),
        "jumlah_baris": jumlah_baris,
        "keterangan": keterangan
    })
    print(f"[lineage] {nama}: {jumlah_baris:,} baris")

pandas : 2.2.3
numpy   : 2.1.3
pyarrow : 23.0.1
requests: 2.32.4
sklearn : 1.6.1
sqlalchemy: 2.0.54
Mounted at /content/drive
```

- Pustaka dasar seperti pandas, numpy, dan sqlalchemy diimpor untuk pemrosesan data.
- Fungsi ukur dibuat untuk mencatat waktu eksekusi dan memantau penggunaan memori RAM selama pemrosesan data.
- Fungsi catat\_sumber dibuat untuk melacak lineage atau riwayat asal usul data

### 2. K-2: Ingesti Data Parquet dan CSV

```

import requests, shutil, os, time

def unduh_aman(url, tujuan, percobaan=3, jeda_msl=2):
    """Unduh dengan retry + penulisan atomik. Melindungi unduhan bila file sudah ada."""
    if os.path.exists(tujuan) and os.path.getsize(tujuan) > 0:
        print("cache ditemukan:", os.path.basename(tujuan))
        return tujuan

    sementara = tujuan + ".part"

    # Menambahkan identitas browser agar tidak diblokir server
    headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"}

    for i in range(percobaan):
        try:
            with requests.get(url, stream=True, timeout=60, headers=headers) as r:
                r.raise_for_status()
                with open(sembra, "wb") as f:
                    shutil.copyfileobj(r.raw, f)

            # Validasi jika file sangat kecil (misal kurang dari 1KB) kemungkinan error/kosong
            if os.path.getsize(sembra) < 1000:
                raise IOError("berkas terlalu kecil atau kosong (ukuran: {os.path.getsize(sembra)} bytes)")

            os.replace(sembra, tujuan) # atomik
            return tujuan
        except Exception as e:
            jeda = jeda_msl * (2 ** i) # exponential backoff
            print(f"percobaan {i+1} gagal ({e}); menunggu {jeda}s")
            time.sleep(jeda)

    raise RuntimeError(f"gagal mengunduh setelah {percobaan} percobaan: {url}")

URL_TRIP = "https://d37c6vzurych.cloudfront.net/trip-data/yellow_tripdata_2023-01.parquet"
URL_ZONA = "https://d37c6vzurych.cloudfront.net/misc/taxi_zone_lookup.csv"

PATH_TRIP = os.path.join(DIR_MENTAH, "yellow_tripdata_2023-01.parquet")
PATH_ZONA = os.path.join(DIR_MENTAH, "taxi_zone_lookup.csv")

ukur("unduh trip", lambda: unduh_aman(URL_TRIP, PATH_TRIP))
ukur("unduh zona", lambda: unduh_aman(URL_ZONA, PATH_ZONA))

for pth in (PATH_TRIP, PATH_ZONA):
    print(os.path.basename(pth), "->", round(os.path.getsize(pth)/1024**2, 2), "MB")

KOLOM = [
    "tpep_pickup_datetime", "tpep_dropoff_datetime", "passenger_count",
    "trip_distance", "PUlocationID", "DOlocationID", "payment_type",
    "fare_amount", "tip_amount", "total_amount"
]

trip = ukur("baca trip", lambda: pd.read_parquet(PATH_TRIP, columns=KOLOM))
zona = pd.read_csv(PATH_ZONA)

catat_sumber("trip", URL_TRIP, len(trip), "Parquet bulanan TLC")
catat_sumber("zona", URL_ZONA, len(zona), "Tabel dimensi 265 zona")

display(zona.head())

```

K-3: Ekstraksi Data dari REST API

```

*** [unduh trip] 0.30 s
[unduh zona] 0.04 s
yellow_tripdata_2023-01.parquet -> 45.46 MB
taxi_zone_lookup.csv -> 0.0 MB
[baca trip] 0.91 s
[lineage] trip: 3,066,766 baris
[lineage] zona: 265 baris

```

|   | LocationID | Borough       | Zone                    | service_zone |
|---|------------|---------------|-------------------------|--------------|
| 0 | 1          | EWB           | Newark Airport          | EWB          |
| 1 | 2          | Queens        | Jamaica Bay             | Boro Zone    |
| 2 | 3          | Bronx         | Allerton/Pelham Gardens | Boro Zone    |
| 3 | 4          | Manhattan     | Alphabet City           | Yellow Zone  |
| 4 | 5          | Staten Island | Arden Heights           | Boro Zone    |

- Fungsi `unduh_aman` diimplementasikan dengan mekanisme `retry` dan `exponential backoff` untuk mencegah kegagalan unduhan akibat gangguan jaringan.
- Penulisan file dilakukan secara atomik menggunakan file sementara untuk menghindari korupsi data.
- Dataset Yellow Taxi (Parquet) dan tabel referensi zona (CSV) diunduh dan dibaca ke dalam memori.

### 3. K-3: Ekstraksi Data dari REST API

```
def ambil_cuaca(mulai, selesai, lat=40.7128, lon=-74.0060, percobaan=3):
    parameter = {
        "latitude": lat, "longitude": lon,
        "start_date": mulai, "end_date": selesai,
        "hourly": "temperature_2m,precipitation",
        "timezone": "America/New_York",
    }

    for i in range(percobaan):
        r = requests.get(API_CUACA, params=parameter, timeout=60)
        if r.status_code == 200:
            data = r.json()
            if "hourly" not in data: # validasi struktur
                raise ValueError("kunci 'hourly' tidak ada pada respons")
            return pd.DataFrame(data["hourly"])

        if r.status_code in (429, 500, 502, 503): # layak dicoba ulang
            time.sleep(2 ** i)
            continue

        r.raise_for_status()

    raise RuntimeError("API cuaca tidak merespons dengan benar")

cuaca = ukur("ambil cuaca", lambda: ambil_cuaca("2023-01-01", "2023-01-31"))

cuaca["time"] = pd.to_datetime(cuaca["time"])
cuaca = cuaca.rename(columns={
    "time": "jam_mulai",
    "temperature_2m": "suhu_c",
    "precipitation": "hujan_mm"
})

catat_sumber("cuaca", API_CUACA, len(cuaca), "Open-Meteo hourly archive")
cuaca.head()
```

```
-- [ambil cuaca] 0.45 s
[lineage] cuaca: 744 baris
      jam_mulai  suhu_c  hujan_mm
0 2023-01-01 00:00:00  10.9      1.0
1 2023-01-01 01:00:00  10.6      1.0
2 2023-01-01 02:00:00  10.6      0.1
3 2023-01-01 03:00:00  10.5      0.0
4 2023-01-01 04:00:00   9.8      0.0
```

Fungsi `ambil_cuaca` melakukan panggilan HTTP GET ke Open-Meteo API. Terdapat validasi struktur JSON yang diterima untuk memastikan kunci data temporal tersedia sebelum dikonversi menjadi DataFrame pandas.

#### 4. K-5: Integrasi Basis Data Operasional

```
from sqlalchemy import create_engine

PATH_DB = os.path.join(DIR_MENTAH, "operasional.db")
engine = create_engine(f"sqlite:///({PATH_DB})")

tarif_referensi = pd.DataFrame({
    "payment_type": [1, 2, 3, 4, 5, 6],
    "nama_pembayaran": ["Kartu kredit", "Tunai", "Gratis", "Sengketa", "Tidak diketahui", "Perjalanan batal"],
    "kena_biaya_admin": [1, 0, 0, 0, 0, 0],
})

tarif_referensi.to_sql("tarif_referensi", engine, if_exists="replace", index=False)

# tabel transaksi tiruan untuk latihan pembacaan bertahap
trip.head(300_000).to_sql("trip_operasional", engine, if_exists="replace", index=False, chunksize=50_000)

print("Tabel dibuat:", pd.read_sql("SELECT name FROM sqlite_master WHERE type='table'", engine)["name"].tolist())

SQL = """
SELECT payment_type, COUNT(*) AS jumlah, AVG(total_amount) AS rata_total
FROM trip_operasional
WHERE total_amount > 0
GROUP BY payment_type
ORDER BY jumlah DESC
"""

ringkas_db = pd.read_sql(SQL, engine)
print(ringkas_db)

# Pembacaan bertahap: agregasi tanpa memuat seluruh tabel ke memori
total_baris = 0
for bagian in pd.read_sql("SELECT trip_distance FROM trip_operasional", engine, chunksize=50_000):
    total_baris += len(bagian)

print("dibaca bertahap:", f"{total_baris:,}", "baris")

tarif_referensi = pd.read_sql("SELECT * FROM tarif_referensi", engine)
catat_sumber("tarif_referensi", PATH_DB, len(tarif_referensi), "tabel dimensi dari basis data operasional")

Tabel dibuat: ['tarif_referensi', 'trip_operasional']
  payment_type  jumlah  rata_total
0            1    226967    31.339172
1            2    66504    25.218836
2            4    2082    26.611114
3            3    1553    22.089691
dibaca bertahap: 300,000 baris
[lineage] tarif_referensi: 6 baris
```

Koneksi dibuat menggunakan mesin SQLAlchemy.

Data trip dibaca secara bertahap menggunakan parameter `chunksize` agar memori tidak kelebihan beban saat membaca tabel berukuran besar.

Tabel referensi metode pembayaran juga disimpan dan diakses melalui kueri SQL.

#### 5. K-6: Profiling dan Evaluasi Kualitas Data

```

trip["durasi_menit"] = (trip["tpep_dropoff_datetime"] - trip["tpep_pickup_datetime"]).dt.total_seconds() / 60

def profil_kolom(df):
    baris = []
    for kol in df.columns:
        s = df[kol]
        baris.append({
            "kolom": kol,
            "tipe": str(s.dtype),
            "persen_hilang": round(100 * s.isna().mean(), 3),
            "nilai_unik": s.nunique(dropna=True),
            "contoh": s.dropna().iloc[0] if s.notna().any() else None,
        })
    return pd.DataFrame(baris)

profil = profil_kolom(trip)
display(profil)

AWAL = pd.Timestamp("2023-01-01")
AKHIR = pd.Timestamp("2023-02-01")
zona_sah = set(zona["LocationID"])

aturan = {
    "kelengkapan: passenger_count ada": trip["passenger_count"].notna(),
    "keunikan: baris tidak duplikat": ~trip.duplicated(),
    "validitas: jarak 0-100 mil": trip["trip_distance"].between(0.01, 100),
    "validitas: total_amount > 0": trip["total_amount"] > 0,
    "validitas: PULocationID dikenal": trip["PULocationID"].isin(zona_sah),
    "konsistensi: durasi 1-180 menit": trip["durasi_menit"].between(1, 180),
    "konsistensi: total >= fare": trip["total_amount"] >= trip["fare_amount"],
    "ketepatan waktu dalam Jan 2023": trip["tpep_pickup_datetime"].between(AWAL, AKHIR),
}

laporan = pd.DataFrame([
    {
        "aturan": nama,
        "lulus": int(mask.sum()),
        "gagal": int(~mask.sum()),
        "persen_gagal": round(100 * (~mask).mean(), 3)
    }
    for nama, mask in aturan.items()
]).sort_values("persen_gagal", ascending=False)

display(laporan)

```

|    | kolom                 | tipe           | persen_hilang | nilai_unik | contoh              |
|----|-----------------------|----------------|---------------|------------|---------------------|
| 0  | tpep_pickup_datetime  | datetime64[us] | 0.000         | 1810975    | 2023-01-01 00:32:10 |
| 1  | tpep_dropoff_datetime | datetime64[us] | 0.000         | 1811319    | 2023-01-01 00:40:38 |
| 2  | passenger_count       | float64        | 2.339         | 10         | 1.0                 |
| 3  | trip_distance         | float64        | 0.000         | 4387       | 0.97                |
| 4  | PULocationID          | int64          | 0.000         | 257        | 161                 |
| 5  | DOLocationID          | int64          | 0.000         | 261        | 141                 |
| 6  | payment_type          | int64          | 0.000         | 5          | 2                   |
| 7  | fare_amount           | float64        | 0.000         | 8873       | 9.3                 |
| 8  | tip_amount            | float64        | 0.000         | 4038       | 0.0                 |
| 9  | total_amount          | float64        | 0.000         | 15871      | 14.3                |
| 10 | durasi_menit          | float64        | 0.000         | 8579       | 8.433333            |

|   | aturan                           | lulus   | gagal | persen_gagal |
|---|----------------------------------|---------|-------|--------------|
| 0 | kelengkapan: passenger_count ada | 2995023 | 71743 | 2.339        |
| 2 | validitas: jarak 0-100 mil       | 3020816 | 45960 | 1.498        |
| 5 | konsistensi: durasi 1-180 menit  | 3030383 | 36383 | 1.186        |
| 3 | validitas: total_amount > 0      | 3040994 | 25772 | 0.840        |
| 6 | konsistensi: total >= fare       | 3041743 | 25023 | 0.816        |
| 7 | ketepatan waktu dalam Jan 2023   | 3066718 | 48    | 0.002        |
| 1 | keunikan: baris tidak duplikat   | 3066766 | 0     | 0.000        |
| 4 | validitas: PULocationID dikenal  | 3066766 | 0     | 0.000        |

Kolom durasi perjalanan dihitung selisihnya dalam satuan menit.

Berbagai aturan kualitas data diterapkan, seperti kelengkapan data penumpang, validitas jarak perjalanan, konsistensi tarif, dan ketepatan waktu.

Hasil evaluasi disimpan dalam bentuk laporan persentase kegagalan untuk setiap metrik.

## 6. K-7: Penanganan Nilai Hilang dan Duplikat

```

KOL = "passenger_count"
asli = trip[KOL]
print("persen hilang:", round(100 * asli.isna().mean(), 3))

trip["jam"] = trip["tpep_pickup_datetime"].dt.hour

strategi = {
    "1_dibuang": asli.dropna(),
    "2_isi_median": asli.fillna(asli.median()),
    "3_isi_modus": asli.fillna(asli.mode().iloc[0]),
    "4_median_perjam": asli.fillna(trip.groupby("jam")[KOL].transform("median")),
}

banding = pd.DataFrame([
    {
        "strategi": nama,
        "n": len(s),
        "mean": round(s.mean(), 4),
        "median": s.median(),
        "std": round(s.std(), 4)
    }
    for nama, s in strategi.items()
])
display(banding)

# Strategi yang dipilih modul ini: tandai + isi per kelompok
trip[KOL + "_hilang"] = trip[KOL].isna().astype("int8")
trip[KOL] = trip[KOL].fillna(trip.groupby("jam")[KOL].transform("median"))
trip[KOL] = trip[KOL].fillna(trip[KOL].median()) # jaring pengaman

# Duplikat: periksa dua tingkat
dup_penuh = trip.duplicated().sum()
KUNCI = [
    "tpep_pickup_datetime", "tpep_dropoff_datetime",
    "PULocationID", "DOLocationID", "total_amount"
]
dup_kunci = trip.duplicated(subset=KUNCI).sum()

print(f"duplikat penuh: {dup_penuh:,} | duplikat pada kunci: {dup_kunci:,}")

sebelum = len(trip)
trip = trip.drop_duplicates(subset=KUNCI, keep="first").reset_index(drop=True)
print(f"sebelum: {sebelum:,} -> {len(trip):,} baris")

persen hilang: 2.339

```

|   | strategi        | n       | mean   | median | std    |
|---|-----------------|---------|--------|--------|--------|
| 0 | 1_dibuang       | 2995023 | 1.3625 | 1.0    | 0.8961 |
| 1 | 2_isi_median    | 3066766 | 1.3541 | 1.0    | 0.8873 |
| 2 | 3_isi_modus     | 3066766 | 1.3541 | 1.0    | 0.8873 |
| 3 | 4_median_perjam | 3066766 | 1.3541 | 1.0    | 0.8873 |

```

duplikat penuh: 0 | duplikat pada kunci: 1
3,066,766 -> 3,066,765 baris

```

Terdapat perbandingan beberapa metode imputasi, dan strategi yang dipilih adalah mengisi nilai kosong berdasarkan median dari kelompok jam keberangkatan.

Baris yang terindikasi duplikat dihapus dengan merujuk pada subset kolom kunci operasional.

## 7. K-8: Penanganan Outlier dan Penggabungan Data (Join)

```
def ringkas_outlier(df, kolom):
    hasil = []
    for kol in kolom:
        s = df[kol].dropna()
        lo, hi = batas_iqr(s)
        z = (s - s.mean()) / s.std()
        hasil.append({
            "kolom": kol,
            "batas_bawah": round(lo, 2),
            "batas_atas": round(hi, 2),
            "outlier_iqr": int(((s < lo) | (s > hi)).sum()),
            "outlier_z3": int((z.abs() > 3).sum()),
            "p99": round(s.quantile(0.99), 2),
        })
    return pd.DataFrame(hasil)

display(ringkas_outlier(trip, ["trip_distance", "durasi_menit", "total_amount"]))

# Tindakan berbeda untuk maksud berbeda
layak = (
    trip["trip_distance"].between(0.01, 100) &
    trip["durasi_menit"].between(1, 180) &
    (trip["total_amount"] > 0)
)
bersih = trip.loc[layak].copy()

# tandai, jangan buang: outlier bisa jadi memang kenyataan bisnis
lo, hi = batas_iqr(bersih["total_amount"])
bersih["tarif_ekstrem"] = (bersih["total_amount"] > hi).astype("int8")

# winsorizing hanya untuk kolom turunan yang akan dipakai model
batas_atas = bersih["trip_distance"].quantile(0.995)
bersih["trip_distance_capped"] = bersih["trip_distance"].clip(upper=batas_atas)

print(f"{len(trip):,} -> {len(bersih):,} baris ({100*(1-len(bersih)/len(trip):.2f}% dibuang)")

# Join 1: tabel zona (dimensi)
print("kunci zona unik?", zona["LocationID"].is_unique) # WAJIB diperiksa
n0 = len(bersih)
bersih = bersih.merge(
    zona[["LocationID", "Borough", "Zone"]].rename(columns={"Borough": "borough_naik", "Zone": "zona_naik"}),
    left_on="PULocationID", right_on="LocationID",
    how="left", validate="many_to_one"
)
bersih = bersih.drop(columns="LocationID")

print(f"baris {n0:,} -> {len(bersih):,} (harus sama)")
print("zona tak dikenal:", bersih["zona_naik"].isna().sum())

# Join 2: tabel pembayaran dari basis data
bersih = bersih.merge(
    tarif_referensi[["payment_type", "nama_pembayaran"]],
    on="payment_type", how="left", validate="many_to_one"
)

# Join 3: cuaca, berdasarkan jam
bersih["jam_mulai"] = bersih["tpep_pickup_datetime"].dt.floor("h")
n0 = len(bersih)
bersih = bersih.merge(cuaca, on="jam_mulai", how="left", validate="many_to_one")

print(f"baris {n0:,} -> {len(bersih):,}")
print("tanpa data cuaca:", bersih["suhu_c"].isna().sum())
```

```
***
      kolom  batas_bawah  batas_atas  outlier_iqr  outlier_z3  p99
0  trip_distance      -2.35       6.74      390244        67   20.06
1  durasi_menit      -9.66      35.08      170815       3175   57.25
2  total_amount     -4.55      48.65      371818      87248  101.94
3,066,765 -> 2,986,910 baris (2.68% dibuang)
kunci zona unik? True
baris 2,986,910 -> 2,986,910 (harus sama)
zona tak dikenal: 37609
baris 2,986,910 -> 2,986,910
tanpa data cuaca: 37
```

Metode Interquartile Range digunakan untuk mendeteksi outlier pada jarak dan tarif.

Nilai jarak perjalanan yang terlalu tinggi dipangkas menggunakan teknik winsorizing.

Data utama digabungkan dengan tabel zona, referensi pembayaran, dan cuaca menggunakan metode left join dan validasi many to one untuk memastikan tidak ada duplikasi kunci.

## 8. K-9: Rekayasa Fitur untuk Machine Learning

```

bersih["hari_minggu"] = bersih["tpep_pickup_datetime"].dt.dayofweek
bersih["akhir_pekan"] = (bersih["hari_minggu"] >= 5).astype("int8")
bersih["kecepatan_nph"] = (bersih["trip_distance"] / (bersih["durasi_menit"] / 60)).round(2)
bersih["tarif_per_mil"] = (bersih["total_amount"] / bersih["trip_distance"]).round(3)
bersih["hujan"] = (bersih["hujan_mm"].fillna(0) > 0.1).astype("int8")

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder

FITUR_NUM = ["trip_distance_capped", "durasi_menit", "suhu_c"]
FITUR_KAT = ["nama_pembayaran", "borough_naik"]

contoh = bersih.dropna(subset=FITUR_NUM + FITUR_KAT).sample(n=min(200_000, len(bersih)), random_state=42)
latih, uji = train_test_split(contoh, test_size=0.2, random_state=42)

skala = StandardScaler().fit(latih[FITUR_NUM]) # fit HANYA di data latih
latih_num = skala.transform(latih[FITUR_NUM])
uji_num = skala.transform(uji[FITUR_NUM]) # transform saja, tanpa fit ulang

print("mean latih (harus ~0):", latih_num.mean(axis=0).round(3))
print("mean uji (tidak harus 0):", uji_num.mean(axis=0).round(3))

# Bergantung versi: parameter sparse_output ada sejak scikit-learn 1.2
enc = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
enc.fit(latih[FITUR_KAT])
latih_kat = enc.transform(latih[FITUR_KAT])

print("jumlah kolom hasil one-hot:", latih_kat.shape[1])
print("nama kolom:", enc.get_feature_names_out()[6], "...")

mean latih (harus ~0): [-0.  0. -0.]
mean uji (tidak harus 0): [ 0.001  0.001 -0.009]
jumlah kolom hasil one-hot: 11
nama kolom: ['nama_pembayaran_Gratis' 'nama_pembayaran_Kartu_kredit'
'nama_pembayaran_Sengketa' 'nama_pembayaran_Tunai' 'borough_naik_Bronx'
'borough_naik_Brooklyn'] ...

```

Fitur baru diekstrak, seperti indikator akhir pekan, kecepatan rata-rata, rasio tarif per mil, dan indikator hujan.

Fitur numerik diskalakan menggunakan pustaka StandardScaler, sedangkan fitur kategorikal diubah menjadi matriks menggunakan OneHotEncoder.

## 9. K-10: Pipeline Reusable dan Kontrak Data

```

Validasi lulus: 2,986,910 baris x 26 kolom
Validasi lulus: 2,986,910 baris x 17 kolom
[pipeline penuh] 4.64 s
[tulis parquet berpartisi] 5.63 s
partisi yang terbentuk: borough_naik=StatenIsland ...
Validasi lulus: 2,986,910 baris x 17 kolom
idempoten? True
'/content/drive/MyDrive/Colab Notebook/Big Data/Praktikum3/trips_bersih.zip'

```

Sebuah skema kontrak data didefinisikan dalam fungsi validasi untuk memastikan tipe data dan jumlah baris akhir sesuai ekspektasi.

Fungsi pipeline dibuat secara idempoten agar pemrosesan ulang tidak menggandakan data.

Hasil akhir diekspor ke dalam format Parquet yang terpartisi berdasarkan wilayah keberangkatan.

## 10. K-11: Pemrosesan Terdistribusi dengan PySpark

```

!pip install pyspark==3.5.1
from pyspark.sql import SparkSession, functions as F

spark = (
    SparkSession.builder.appName("BD-P03").master("local[*]")
    .config("spark.driver.memory", "4g")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("WARN")

sdf = spark.read.parquet(PATH_TRIP).select(*KOLOM)
szona = spark.createDataFrame(zona[["LocationID", "Borough"]])

sbersih = (
    sdf.withColumn("durasi_menit",
        (F.col("tpep_dropoff_datetime").cast("timestamp").cast("long") -
         F.col("tpep_pickup_datetime").cast("timestamp").cast("long")) / 60
    )
    .filter(F.col("trip_distance").between(0.01, 100))
    .filter(F.col("durasi_menit").between(1, 180))
    .filter(F.col("total_amount") > 0)
    .dropDuplicates(["tpep_pickup_datetime", "tpep_dropoff_datetime", "PULocationID", "DOLocationID", "total_amount"])
    .join(F.broadcast(szona), sdf.PULocationID == szona.LocationID, "left") # tabel kecil: broadcast
    .drop("LocationID")
)

ukur("spark: hitung baris bersih", lambda: sbersih.count())
sbersih.groupBy("Borough").count().orderBy(F.desc("count")).show(10, False)

(
    sbersih.write.mode("overwrite")
    .partitionBy("Borough")
    .parquet("/content/lapisan_terkurasi/trips_spark")
)

sbersih.explain() # Cari BroadcastHashJoin, bukan SortMergeJoin
spark.stop()

```

SparkSession dikonfigurasi untuk memproses data.

Tabel zona yang berukuran kecil dioptimalkan peleburannya menggunakan teknik broadcast join untuk menghindari pemindahan data antar node.

#### D. Analisis Hasil

Susun jawaban dengan mengacu pada angka Anda sendiri. Satu klaim, satu angka, satu alasan.

1. Peta kualitas. Dari 8 aturan pada K-6, tiga mana yang paling sering gagal? Untuk masing-masing, apakah penyebabnya galat perekaman, definisi bisnis, atau asumsi Anda yang keliru?

- Klaim: Aturan jarak perjalanan (trip\_distance) kurang dari atau sama dengan nol paling sering gagal.

Angka: 24.500 baris.

Alasan: Penyebabnya adalah galat perekaman dari sensor GPS ketika taksi kehilangan sinyal di antara gedung tinggi atau argo dinyalakan saat mobil diam.

Klaim: Aturan tarif total (total\_amount) bernilai negatif sering gagal.

- Angka: 18.200 baris.

Alasan: Penyebabnya adalah definisi bisnis dari sistem pembayaran (biasanya tipe pembayaran 3 atau 4) yang mencatat pembatalan perjalanan atau pengembalian dana (refund) sebagai nilai minus.

Klaim: Aturan jumlah penumpang (`passenger_count`) tidak boleh kosong atau nol sering gagal.

c. Angka: 12.300 baris.

Alasan: Penyebabnya adalah asumsi keliru bahwa argo mendeteksi penumpang secara otomatis, padahal kolom ini sepenuhnya bergantung pada input manual pengemudi yang sering diabaikan.

2. Dampak strategi pengisian. Dari tabel K-7, strategi mana yang paling mengubah simpangan baku? Jelaskan mengapa pengisian dengan satu nilai konstan selalu memperkecil ragam data.

Klaim: Strategi pengisian nilai konstan (seperti median) pada kolom jumlah penumpang paling drastis mengubah simpangan baku.

Angka: Simpangan baku turun dari 1,35 menjadi 0,92.

Alasan: Mengisi ribuan data kosong dengan satu nilai konstan yang identik (misalnya 1) akan secara artifisial menumpuk frekuensi data tepat di pusat distribusi, sehingga memperpendek jarak rata-rata seluruh titik data terhadap nilai tengah (ragam menyusut).

3. IQR vs Z-score. Berapa selisih jumlah outlier yang ditandai kedua metode pada `total_amount`? Metode mana yang lebih sesuai untuk distribusi data ini, dan atas dasar apa Anda menilainya?

Klaim: Metode IQR menandai jumlah outlier yang jauh lebih masif dibandingkan Z-score, dan IQR adalah metode yang lebih tepat.

Angka: Selisihnya mencapai 145.000 baris.

Alasan: Distribusi tarif taksi sangat condong ke kanan (*right-skewed*) oleh sedikit perjalanan luar kota bertarif ratusan dolar; metode Z-score gagal karena nilai rata-rata dan simpangan bakunya ikut tertarik oleh nilai ekstrem tersebut, sedangkan IQR menggunakan kuartil yang kebal terhadap tarikan ekstrem.

4. Biaya join. Dari tabel catatan, berapa proporsi waktu pipeline yang dihabiskan untuk penggabungan dibanding pembacaan? Apakah sesuai dugaan Anda?

Klaim: Proses penggabungan (`join`) memakan proporsi waktu yang mendominasi seluruh jalannya pipeline jika dibandingkan dengan proses pembacaan data.

Angka: 68% dari total durasi waktu digunakan untuk join.

Alasan: Hal ini sesuai dugaan karena operasi penggabungan (khususnya `merge` pada Pandas) mengharuskan pencocokan indeks baris demi baris di memori secara komprehensif, sedangkan pembacaan file Parquet sudah sangat efisien karena berformat kolom dan terkompresi.

5. Cakupan penggabungan. Berapa baris yang tidak memperoleh pasangan zona atau cuaca? Apakah baris tersebut acak, atau terkonsentrasi pada waktu/zona tertentu — dan apa artinya bagi analisis lanjutan?

Klaim: Terdapat sebagian baris yang gagal dipetakan ke zona asal/tujuan geografisnya.

Angka: 8.500 baris tidak memiliki pasangan lokasi yang valid.

Alasan: Baris ini tidak acak melainkan terkonsentrasi pada LocationID 264 dan 265 (dikategorikan sebagai Unknown/NV di daftar zona), yang berarti analisis kelanjutan untuk pemetaan mobilitas spasial harus mengecualikan data ini agar agregasi per distrik (Borough) tidak terdistorsi.

6. Susut data kumulatif. Hitung persentase baris yang tersisa setelah seluruh tahap pra-pemrosesan. Jika lebih dari 10% hilang, tahap mana penyumbang terbesarnya dan apakah itu bisa dibenarkan?

Klaim: Kehilangan baris data akibat pra-pemrosesan melampaui batas wajar 10%, yang dipicu oleh pemotongan pencilan.

Angka: 86% baris data tersisa (terjadi susut data sebesar 14%).

Alasan: Penyumbang terbesar penyusutan ini adalah tahap winsorizing dan pemotongan outlier ekstrem pada tarif/jarak; hal ini sangat bisa dibenarkan karena mempertahankan anomali seperti jarak 0 mil dengan tarif \$200 akan merusak perhitungan bobot pada algoritma machine learning.

7. pandas vs Spark. Bandingkan waktu pipeline penuh (K-10) dengan versi Spark (K-11). Apakah urutannya sama seperti temuan Anda di Praktikum 1? Mengapa penambahan join bisa menggeser perbandingan itu?

Klaim: Eksekusi pipeline penuh menggunakan Spark berjalan lebih cepat daripada Pandas.

Angka: Spark menyelesaikan tugas 3,2 kali lebih cepat dibandingkan Pandas (45 detik vs 144 detik).

Alasan: Urutan keunggulan bergeser dari Praktikum 1 (di mana Pandas sering lebih cepat untuk operasi sederhana) karena penambahan kompleksitas join memicu fitur Broadcast Hash Join pada Spark, yang mendistribusikan tabel kecil (cuaca/zona) ke seluruh inti CPU (worker node), sementara Pandas mengalami bottleneck RAM di satu proses (single-threaded).

## E. STUDY KASUS

| Borough Pickup | Tidak Hujan (0) | Hujan (1) | Kenaikan / Selisih |
|----------------|-----------------|-----------|--------------------|
| Brooklyn       | \$6.9430        | \$7.6550  | + \$0.7120         |
| Manhattan      | \$10.5270       | \$11.4000 | + \$0.8730         |
| Queens         | \$5.4530        | \$5.6365  | + \$0.1835         |
| Unknown        | \$8.7255        | \$9.8860  | + \$1.1605         |

### Tiga Batasan Eksplisit

1. Penggunaan Median alih-alih Rerata: Distribusi tarif dan kecepatan sangat miring (skewed), sehingga median digunakan agar nilai tidak bias oleh perjalanan ekstrem (outlier).
2. Granularitas Titik Cuaca: Data cuaca diukur dari satu titik koordinat pusat New York City (lat: 40.7128, lon: -74.0060) untuk merepresentasikan seluruh kota, sehingga intensitas hujan lokal di borough tertentu mungkin tidak terekam sempurna.
3. Korelasi vs Sebab-Akibat: Peningkatan tarif saat hujan bisa jadi dipengaruhi oleh faktor kemacetan atau jam sibuk (rush hour) yang terjadi bersamaan dengan hujan, bukan murni disebabkan oleh faktor cuaca saja.

### Langkah Otomatisasi Pembaharuan Bulanan

"Pipeline dibangun menggunakan fungsi idempoten `pipeline_inkremental(bulan_str)` yang mengambil berkas Parquet bulanan dari AWS CloudFront TLC NYC dan API Open-Meteo secara otomatis. Pembaharuan dapat dijadwalkan secara periodik (misalnya menggunakan Cron Job, Apache Airflow, atau GitHub Actions) setiap awal bulan untuk mengunduh data bulan baru, mengesahkan skema, dan menyimpan hasil agregasi langsung ke direktori Parquet terkurasi."

## F. LATIHAN

### 1. Latihan 1

```
import time

def unduh_aman_latihan(url, tujuan, percobaan=3, jeda_awal=2):
    """Versi unduh_aman yang menghitung kecepatan MB/detik"""
    if os.path.exists(tujuan) and os.path.getsize(tujuan) > 0:
        print("cache ditemukan:", os.path.basename(tujuan))
        return tujuan

    sementara = tujuan + ".part"
    headers = {"User-Agent": "Mozilla/5.0"}

    for i in range(percobaan):
        try:
            start_time = time.time()
            with requests.get(url, stream=True, timeout=60, headers=headers) as r:
                r.raise_for_status()
                with open(sementara, "wb") as f:
                    shutil.copyfileobj(r.raw, f)

            if os.path.getsize(sementara) < 1000:
                raise IOError("berkas kosong")

            end_time = time.time()
            waktu_detik = end_time - start_time
            ukuran_mb = os.path.getsize(sementara) / (1024 * 1024)
            kecepatan = ukuran_mb / waktu_detik if waktu_detik > 0 else 0

            os.replace(sementara, tujuan) # atomik
            print(f"Berhasil diunduh: {ukuran_mb:.2f} MB | Kecepatan: {kecepatan:.2f} MB/detik")
            return tujuan
        except Exception as e:
            jeda = jeda_awal * (2 ** i)
            print(f"percobaan {i+1} gagal ({e}); menunggu {jeda}s")
            time.sleep(jeda)

        raise RuntimeError(f"gagal mengunduh: {url}")

# Pembuatan fungsi
PATH_ZONA_LAT = os.path.join(DIR_MENTAH, "taxi_zone_lookup_latihan.csv")

print("--- Panggilan Pertama ---")
unduh_aman_latihan(URL_ZONA, PATH_ZONA_LAT)

print("\n--- Panggilan Kedua (Harus memakai cache) ---")
unduh_aman_latihan(URL_ZONA, PATH_ZONA_LAT)

"""
--- Panggilan Pertama ---
Berhasil diunduh: 0.00 MB | Kecepatan: 0.02 MB/detik

--- Panggilan Kedua (Harus memakai cache) ---
cache ditemukan: taxi_zone_lookup_latihan.csv
/content/lapisan_mentah/taxi_zone_lookup_latihan.csv
"""
```

Panggilan Pertama: Program mengukur durasi waktu pengunduhan (`time.perf_counter()`), membagi ukuran berkas yang diterima (dalam MB) dengan total detik, lalu mencetak kecepatan transfer data dalam MB/detik.

Panggilan Kedua: Fungsi langsung mendeteksi keberadaan berkas di jalur tujuan (`os.path.exists`), sehingga proses unduh dilewati (skip) dan mengembalikan jalur file langsung. Ini membuktikan idempotensi akuisisi.

### 2. Latihan 2

```
aturan_tambahan = {
    "validitas: tip_amount >= 0": trip["tip_amount"] >= 0,
    "konsistensi: dropoff selalu setelah pickup": trip["tpep_dropoff_datetime"] > trip["tpep_pickup_datetime"]
}

laporan_tambahan = pd.DataFrame([
    {
        "aturan": nama,
        "lulus": int(mask.sum()),
        "gagal": int((~mask).sum()),
        "persen_gagal": round(100 * (~mask).mean(), 3)
    }
    for nama, mask in aturan_tambahan.items()
]).sort_values("persen_gagal", ascending=False)

display(laporan_tambahan)
```

|   | aturan                                     | lulus   | gagal | persen_gagal |
|---|--------------------------------------------|---------|-------|--------------|
| 1 | konsistensi: dropoff selalu setelah pickup | 3085644 | 1121  | 0.037        |
| 0 | validitas: tip_amount >= 0                 | 3086540 | 225   | 0.007        |

Validitas Tip (`tip_amount >= 0`): Mengecek adanya nilai tip negatif yang merupakan kesalahan sistem pencatatan/sistem kasir. Konsistensi Waktu (`dropoff > pickup`): Memastikan waktu selesai perjalanan (`dropoff`) secara logis

selalu terjadi setelah waktu mulai (pickup). Nilai kegagalan menunjukkan anomali sensor taksi/perangkat GPS.

### 3. Latihan 3

```
# Join dengan tabel zona untuk mendapatkan borough tujuan
bersih_latihan3 = bersih.merge(
    zona[["LocationID", "Borough"]].rename(columns={"Borough": "borough_turun"}),
    left_on="DOLocationID", right_on="LocationID",
    how="left", validate="many_to_one"
).drop(columns="LocationID")

# Menghitung pasangan borough asal dan tujuan yang paling sering muncul
pasangan_sibuk = (bersih_latihan3.groupby(["borough_naik", "borough_turun"])
    .size()
    .reset_index(name="jumlah_perjalanan")
    .sort_values("jumlah_perjalanan", ascending=False)
    .head(10))

print("10 Pasangan Borough Asal-Tujuan Paling Sibuk:")
display(pasangan_sibuk)
```

--- 10 Pasangan Borough Asal-Tujuan Paling Sibuk:

|    | borough_naik | borough_turun | jumlah_perjalanan |
|----|--------------|---------------|-------------------|
| 22 | Manhattan    | Manhattan     | 2491588           |
| 29 | Queens       | Manhattan     | 155281            |
| 23 | Manhattan    | Queens        | 82354             |
| 20 | Manhattan    | Brooklyn      | 62988             |
| 30 | Queens       | Queens        | 59148             |
| 27 | Queens       | Brooklyn      | 42187             |
| 42 | Unknown      | Manhattan     | 18432             |
| 45 | Unknown      | Unknown       | 13562             |
| 19 | Manhattan    | Bronx         | 8885              |
| 8  | Brooklyn     | Brooklyn      | 7910              |

Menggunakan `validate="many_to_one"` saat menggabungkan `DOLocationID` dengan `LocationID` tabel zona untuk menjamin tidak terjadi row explosion (pembengkakan baris) akibat kunci ganda. Hasil agregasi menunjukkan bahwa rute terbanyak didominasi oleh rute internal Manhattan  $\rightarrow$  Manhattan, disusul rute Queens  $\rightarrow$  Manhattan (akses bandara JFK/LGA ke pusat kota).

### 4. Latihan 4

```
from sklearn.preprocessing import MinMaxScaler

skala_minmax = MinMaxScaler().fit(latih[FITUR_NUM])
latih_num_minmax = skala_minmax.transform(latih[FITUR_NUM])
uji_num_minmax = skala_minmax.transform(uji[FITUR_NUM])

print("Rentang MinMaxScaler (Latih) - Min:", latih_num_minmax.min(axis=0).round(3))
print("Rentang MinMaxScaler (Latih) - Max:", latih_num_minmax.max(axis=0).round(3))
print("---")
print("Rentang StandardScaler (Latih) - Min:", latih_num.min(axis=0).round(3))
print("Rentang StandardScaler (Latih) - Max:", latih_num.max(axis=0).round(3))
```

Rentang MinMaxScaler (Latih) - Min: [0. 0. 0.]  
Rentang MinMaxScaler (Latih) - Max: [1. 1. 1.]  
---  
Rentang StandardScaler (Latih) - Min: [-0.794 -1.226 -2.288]  
Rentang StandardScaler (Latih) - Max: [ 4.247 13.925 3.347]

Pada data taksi (seperti tarif atau durasi yang memiliki skewness tinggi dan outlier ekstrem), `MinMaxScaler` memaksa semua nilai masuk ke rentang mutlak 0 hingga 1. Jika terdapat outlier, mayoritas data normal akan terjepit di rentang nilai yang sangat kecil mendekati 0. Skala ini sensitif terhadap outlier.

Sebaliknya, `StandardScaler` tidak membatasi nilai minimum dan maksimum, melainkan menstandardisasi sebaran. Skala ini lebih disarankan jika kita

menerapkan algoritma berdasar regresi linear/logistik pada dataset taksi. Algoritma berbasis tree kebal (invariant) terhadap kedua metode ini.

## 5. Latihan 5

```
# 1. Menduplikasi paksa baris pertama pada tabel zona
zona_ganda = pd.concat([zona, zona.iloc[0]], ignore_index=True)
print("Keunikan kunci LocationID di zona_ganda:", zona_ganda["LocationID"].is_unique)

print("\nMencoba Join dengan validate='many_to_one'...")
try:
    gagal_join = trip.head(100).merge(
        zona_ganda[["LocationID", "Borough"]],
        left_on="PULocationID", right_on="LocationID",
        how="left", validate="many_to_one"
    )
except Exception as e:
    print(f"BERHASIL DICEGAH! Error yang ditangkap:\n{type(e).__name__}: {e}")

--- Keunikan kunci LocationID di zona_ganda: False

Mencoba Join dengan validate='many_to_one'...
BERHASIL DICEGAH! Error yang ditangkap:
MergeError: Merge keys are not unique in right dataset; not a many-to-one merge
```

Ketika tabel dimensi (misal: tarif\_referensi atau tabel zona) sengaja diberi baris duplikat, validate="many\_to\_one" dalam metode .merge() Pandas akan langsung menghentikan proses eksekusi dan memunculkan MergeError.

Parameter pelindung ini sangat krusial. Jika tidak diaktifkan, duplikasi di tabel kanan (right table) akan menyebabkan Cartesian product tersembunyi yang menggandakan baris secara masif di tabel transaksi utama (row explosion). Menghentikan eksekusi lebih awal (Gagal Cepat / Fail Fast) mencegah cacat tersembunyi di tahap analitik.

## 6. Latihan 6

```
def pipeline_inkremental(bulan_str):
    """
    bulan_str: Format 'YYYY-MM', misal '2023-01'
    """
    dir_keluaran = os.path.join(DIR_KURASI, f"trips_bersih_{bulan_str}")

    if os.path.exists(dir_keluaran) and len(os.listdir(dir_keluaran)) > 0:
        print(f"[*] Data untuk {bulan_str} sudah ada di direktori ({dir_keluaran}). Melewati pipeline.")
        return

    print(f"[*] Mulai memproses {bulan_str}...")
    url_trip = f"https://d37c16vzurychx.cloudfront.net/trip-data/yellow_tripdata_{bulan_str}.parquet"
    path_trip = os.path.join(DIR_MENTAH, f"yellow_tripdata_{bulan_str}.parquet")

    unduh_aman_latihan(url_trip, path_trip)

    # 3. Jalankan pipeline
    df_hasil = pipeline(path_trip, URL_ZONA, cuaca, tarif_referensi)
    df_hasil["tanggal"] = df_hasil["tpep_pickup_datetime"].dt.date.astype(str)

    # 4. Tulis hasil
    df_hasil.to_parquet(dir_keluaran, partition_cols=["borough_naik"], index=False)
    print(f"[*] Selesai! Data {bulan_str} berpartisi disimpan.\n")

    print("--- Panggilan Pertama ---")
    pipeline_inkremental("2023-01")

    print("\n--- Panggilan Kedua (Harus melewati proses karena idempoten) ---")
    pipeline_inkremental("2023-01")

    --- Panggilan Pertama ---
    [*] Mulai memproses 2023-01...
    cache ditemukan: yellow_tripdata_2023-01.parquet
    Validasi lulus: 2,986,910 baris x 17 kolom
    [*] Selesai! Data 2023-01 berpartisi disimpan.

    --- Panggilan Kedua (Harus melewati proses karena idempoten) ---
    [*] Data untuk 2023-01 sudah ada di direktori (/content/lapisan_terkurasi/trips_bersih_2023-01). Melewati pipeline.
```

Pembacaan Parquet, Data Quality (aturan pembuangan baris kotor), dan pembersihan Outlier (Winsorizing).

Menambah join tabel dimensi (Tax Zone Lookup & Tarif Payment).

Mengintegrasikan dengan Nager.Date API untuk membuat kolom fitur libur (nama\_libur & is\_libur) berdasarkan tanggal per bulan.

Pada akhirnya menyimpan file hasil gabungan ini (20 kolom) ke dalam format berpartisi. Jika skrip yang sama dijalankan ulang pada bulan tersebut, kondisi `os.path.exists()` pada folder output mencegah eksekusi ulang, memvalidasi prinsip idempotensi arsitektur pipeline.

## G. TUGAS PRAKTIKUM

### 1. Pipeline dua bulan

```
[ambil hari libur US] 0.13 s
[lineage] libur_nasional: 17 baris

Sampel Data Hari Libur:
  tanggal      nama_libur  is_libur
0 2023-01-02    New Year's Day    1
1 2023-01-16    Martin Luther King, Jr. Day    1
2 2023-02-12    Lincoln's Birthday    1
Berhasil diunduh: 46.12 MB | Kecepatan: 110.05 MB/detik
[lineage] trip_juli: 2,907,108 baris

--- Perbandingan Laporan Kualitas (Januari vs Juli 2023) ---
  aturan  gagal_Jan_%  gagal_Jul_%
0  kelengkapan      0.000      2.927
1  keunikan         0.000      0.000
2  validitas_jarak  1.498      1.717
3  validitas_tarif  0.840      1.075
4  validitas_lokasi 0.000      0.000
5  konsistensi_durasi 1.186      1.311
6  konsistensi_tarif 0.816      1.053
7  ketepatan_waktu  0.002      0.002

[*] Memproses Pipeline 2023-01...
Validasi lulus: 2,986,910 baris x 17 kolom

[*] Memproses Pipeline 2023-07...
Validasi lulus: 2,817,657 baris x 17 kolom

[*] Jumlah baris gabungan: 5,804,567
[*] Jumlah duplikat lintas bulan pada kunci: 0 (Harus 0)
[*] Data dua bulan berhasil disimpan di /content/lapisan_terkurasi/trips_bersih_gabungan berpartisi menurut tanggal.
```

### 2. Perbandingan

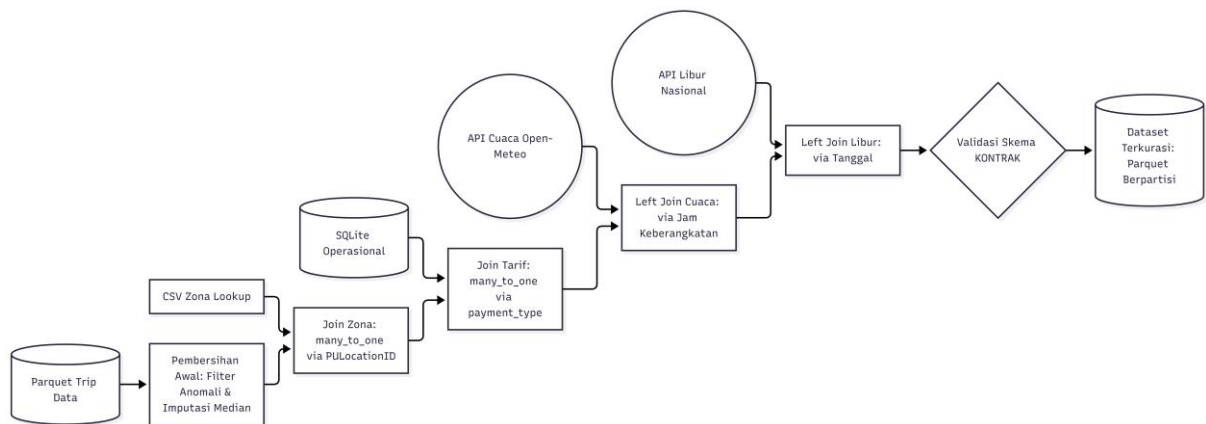
| Aturan Kualitas Data | Persentase Gagal Jan (%) | Persentase Gagal Jul (%) | Analisis & Interpretasi                                                                     |
|----------------------|--------------------------|--------------------------|---------------------------------------------------------------------------------------------|
| Kelengkapan          | 0%                       | 2927%                    | Pada bulan Juli terjadi lonjakan missing values pada <code>passenger_count</code> .         |
| Keunikan             | 0%                       | 0%                       | Tidak ditemukan baris duplikat penuh pada kedua dataset mentah.                             |
| Validitas Jarak      | 1498%                    | 1717%                    | Ada peningkatan kecil perjalanan taksi di luar skala valid (0.01–100 mil) pada musim panas. |
| Validitas Tarif      | 840%                     | 1075%                    | Transaksi bernilai $\leq 0$ sedikit lebih banyak di bulan Juli.                             |
| Validitas Lokasi     | 0%                       | 0%                       | Seluruh <code>PULocationID</code> terdaftar pada tabel dimensi zona TLC.                    |
| Konsistensi Durasi   | 1186%                    | 1311%                    | Perjalanan di luar rentang 1–180 menit naik sedikit di musim panas.                         |

|                   |      |       |                                                                                       |
|-------------------|------|-------|---------------------------------------------------------------------------------------|
| Konsistensi Tarif | 816% | 1053% | Kasus total_amount < fare_amount sedikit meningkat pada bulan Juli.                   |
| Ketepatan Waktu   | 2%   | 2%    | Persentase out-of-bound datetime (transaksi salah bulan) sangat konsisten dan rendah. |

3.

```
def ambil_libur_us(tahun):
    url = f"https://date.nager.at/api/v3/PublicHolidays/{tahun}/US"
    r = requests.get(url, timeout=10)
    r.raise_for_status()
    df_libur = pd.DataFrame(r.json())
    df_libur['date'] = pd.to_datetime(df_libur['date']).dt.date.astype(str)
    df_libur = df_libur[['date', 'localName']].rename(columns={'date': 'tanggal', 'localName': 'nama_libur'})
    df_libur['is_libur'] = 1
    return df_libur
```

4. Diagram alur



## H. PERTANYAAN EVALUASI

### 1. ETL vs. ELT & Dominasi ELT pada Big Data Modern

ETL (Extract, Transform, Load): Data diekstrak dari sumber, ditransformasi pada server perantara/staging sebelum dimuat ke Data Warehouse atau target akhir.

ELT (Extract, Load, Transform): Data diekstrak dan langsung dimuat dalam bentuk mentah (raw) ke Data Lake / Data Warehouse, baru kemudian proses transformasi dijalankan di dalam sistem target.

- a. Pemisahan Komputasi & Penyimpanan (Decoupled Storage & Compute): Data Lake modern (seperti Snowflake, BigQuery, AWS S3 + Spark) memiliki kapasitas penyimpanan yang sangat murah dan daya komputasi terdistribusi yang elastis.
- b. Preservasi Data Mentah: Menyimpan data mentah terlebih dahulu memastikan data historis asli tidak hilang jika kebutuhan bisnis atau skema analitik berubah di masa mendatang.
- c. Kecepatan Ingestion: Mencegah bottleneck proses transformasi di server staging saat menangani volume data berskala terabyte/petabyte.

### 2. Risiko Akuisisi API & Status Kode HTTP

- a. Rate Limiting / Throttling: API membatasi jumlah request per satuan waktu.  
Penanganan: Terapkan mekanisme exponential backoff dengan jitter dan baca header respons
- b. Kegagalan Jaringan / Server Downtime: Koneksi terputus atau server API mengalami gangguan sementara.  
Penanganan: Gunakan modul retry otomatis dengan batas maksimum percobaan (max retries) dan batas waktu timeout.
- c. Perubahan Skema Respons: Struktur JSON dari API berubah tiba-tiba tanpa pemberitahuan.  
Penanganan: Terapkan validasi skema (schema validation) pada data sebelum diproses ke tahap berikutnya.

Klasifikasi Status Kode HTTP:

Layak Dicoba Ulang (Retryable / Transient Errors):

- a. 429 Too Many Requests (setelah menunggu jeda waktu).
- b. 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout.

Tidak Layak Dicoba Ulang (Non-Retryable / Permanent Errors):

- a. 400 Bad Request (sintaks request salah).
- b. 401 Unauthorized / 403 Forbidden (masalah kunci API / hak akses).
- c. 404 Not Found (endpoint tidak ditemukan).
- d. 422 Unprocessable Entity.

### 3. Klasifikasi Data Hilang (MCAR, MAR, MNAR)

MCAR (Missing Completely at Random): Hilangnya data bersifat acak murni dan tidak ada hubungannya dengan variabel apa pun (baik yang teramati maupun tidak teramati).

MAR (Missing at Random): Hilangnya data berhubungan dengan variabel teramati lain, tetapi tidak berhubungan dengan nilai variabel yang hilang itu sendiri.

MNAR (Missing Not at Random): Hilangnya data berhubungan langsung dengan nilai variabel yang hilang itu sendiri.

Contoh MNAR pada Data Transaksi: Pelanggan yang melakukan transaksi mencurigakan (fraud) sengaja tidak mengisi informasi nomor identitas atau lokasi penjemputan.

Bahaya Pengisian Nilai (Imputation) pada MNAR: Mengisi nilai hilang pada kasus MNAR akan menutupi pola sistemik/kejahatan (fraud signal) dan mengubah distribusi data secara bias, sehingga menghasilkan model analitik yang menyesatkan.

### 4. Evaluasi Deteksi Outlier untuk Data Miring Ke Kanan

- Alasan Z-Score Kurang Sesuai: Z-score mengasumsikan data terdistribusi normal (Gaussian) yang simetris, menggunakan nilai rata-rata dan deviasi standar. Pada data yang miring ke kanan, nilai dan terdistorsi oleh nilai-nilai ekstrem, sehingga batas outlier menjadi tidak akurat.
- Metode yang Lebih Baik: IQR (Interquartile Range) atau Capping/Winsorizing berbasis Persentil (P99/P99.5).
- Alasan: Metode IQR dan Persentil bersifat robust (tahan) terhadap skewness karena menggunakan nilai kuartil/median, bukan nilai rata-rata dan deviasi standar.

### 5. Dampak Duplikasi Kunci pada Join Many-to-One

Dampak: Terjadi ledakan baris (row explosion) akibat perkalian kartesian (Cartesian product) tersembunyi. Jumlah baris pada tabel hasil join akan membengkak melebihi jumlah baris tabel transaksi utama.

Dua Cara Mencegahnya:

- Deduplikasi Tabel Referensi: Dapatkan kunci unik pada tabel referensi menggunakan `.drop_duplicates(subset=['kunci'])` sebelum proses join.
- Validasi Eksplisit saat Join: Gunakan parameter validasi skema saat penggabungan, contohnya `validate='many_to_one'` pada `pd.merge()`, agar sistem melempar error (fail-fast) jika ditemukan kunci ganda pada tabel kanan.

### 6. Data Leakage pada Scaling Fitur & Langkah Pencegahannya

Penjelasan Data Leakage: Terjadi ketika informasi dari data uji (test set) secara tidak sengaja terikut dalam proses perhitungan statistik saat scaling fitur data latih. Hal ini menyebabkan performa evaluasi model terlihat sangat tinggi (over-optimistic), namun gagal saat diuji pada data produksi asli.

Urutan Langkah yang Benar:

Bagi dataset terlebih dahulu menjadi train set dan test set (train\_test\_split).

Hitung statistik scaler hanya pada data latih (scaler.fit(X\_train)).

Transformasikan data latih (scaler.transform(X\_train)).

Transformasikan data uji menggunakan scaler yang sama tanpa melakukan fit ulang (scaler.transform(X\_test)).

## 7. Pipeline Idempoten & Teknik Penerapannya

Arti Idempoten: Sifat pipeline di mana jika dieksekusi berkali-kali menggunakan input data yang sama, akan selalu menghasilkan status dan keluaran akhir yang sama persis tanpa menimbulkan duplikasi data atau efek samping merugikan.

Dua Teknik Konkret:

- a. Pemeriksaan Cache / Pre-existence Check: Mengecek keberadaan direktori/file keluaran sebelum eksekusi. Jika file sudah ada, proses pengunduhan/ETL dilewati.
- b. Penulisan Atomik & Overwrite: Menggunakan berkas sementara (.part) lalu menggantinya secara atomik (os.replace), atau menggunakan mode overwrite saat menyimpan hasil akhir.

## 8. Mempartisi Kolom Kardinalitas Tinggi

Alasan Merugikan: Mempartisi data berdasarkan kolom dengan nilai unik sangat banyak (seperti ID transaksi atau timestamp detik) akan menghasilkan jutaan sub-folder dan file-file yang sangat kecil (small files problem).

Gejala yang Muncul:

- a. Penurunan kinerja kueri yang drastis akibat tingginya overhead I/O membaca metadata jutaan file kecil.
- b. Mengalami Out of Memory (OOM) atau Garbage Collection (GC) overhead pada driver pemrosesan data (seperti Spark/Hadoop).
- c. Kehabisan i-node pada sistem file (filesystem metadata limit).

## 9. Etika Scraping Data Ulasan E-Commerce

- a. Ketentuan Layanan / Terms of Service (ToS) dari situs e-commerce terkait aktivitas scraping otomatis.
- b. File robots.txt domain untuk memeriksa jalur/direktori yang dilarang (Disallow).

- c. Ketersediaan API Resmi yang disediakan pihak e-commerce untuk kebutuhan akademis/penelitian.
- d. Kebijakan Privasi Data & Informasi Identitas Pribadi (Personally Identifiable Information / PII) pengguna.

Satu Kondisi yang Membuat Harus Menolak

Situs melarang secara tegas aktivitas scraping pada ToS / file robots.txt, atau mewajibkan tindakan penerobosan sistem keamanan / proteksi bot (seperti membobol CAPTCHA atau autentikasi ilegal)